

Sistemas Distribuídos

Contexto / Arquitetura / Aspectos Distribuídos / Resultados

Pedro Saito, João Pedro Sousa e Marcos Henrique

Universidade Federal do Rio de Janeiro

2026-07-08



1. Contexto

Construir um motor de consultas para ambiente distribuídos com foco na execução de consultas SPARQL federadas.

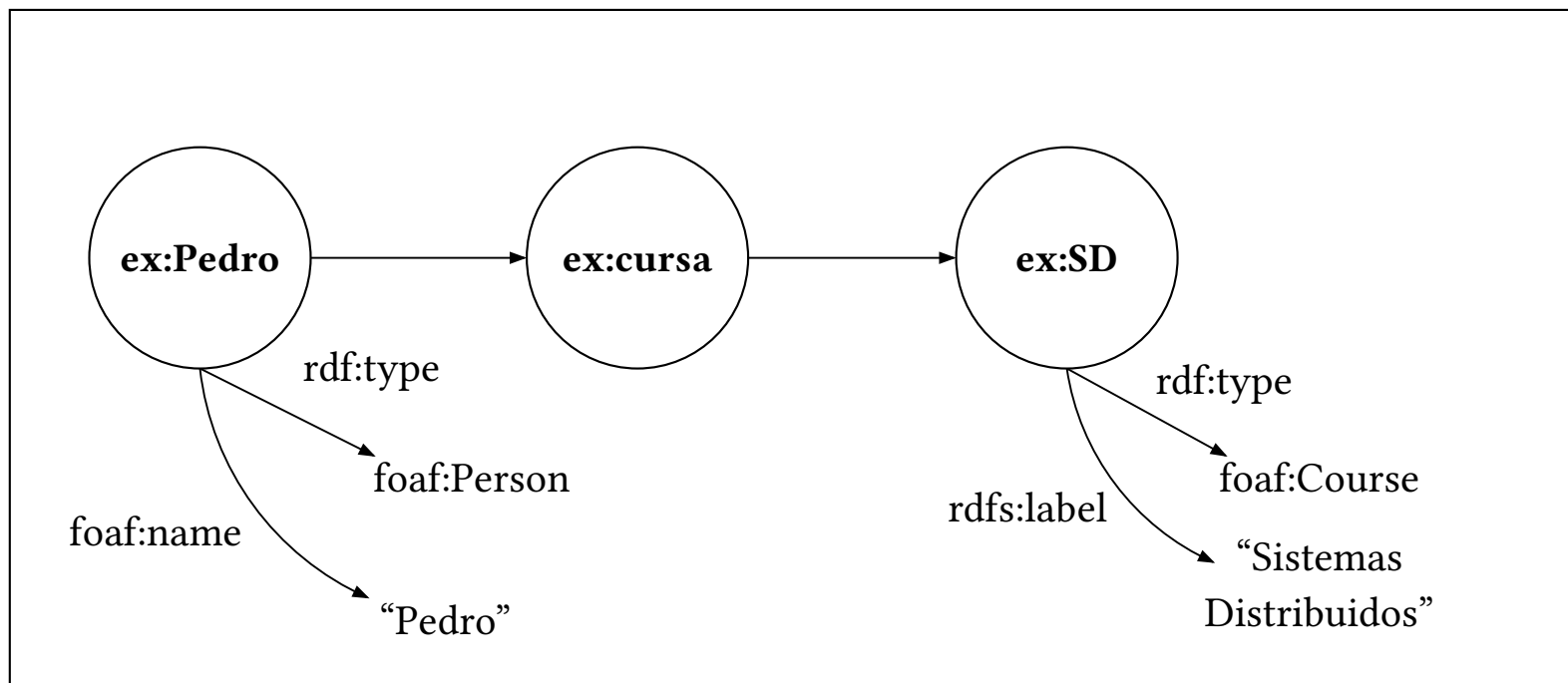


Figura 1: Modelo RDF: recursos conectados por propriedades e valores literais.

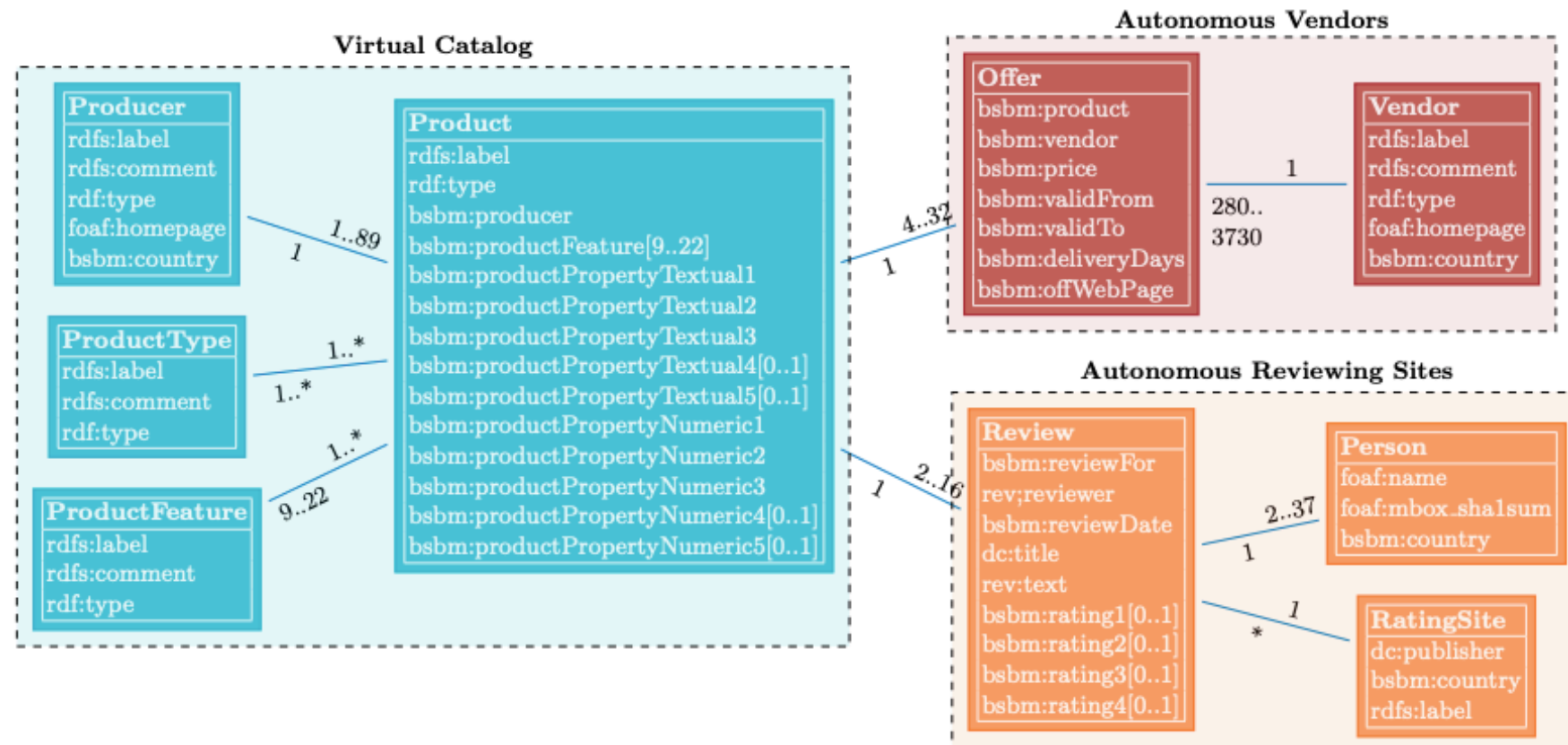


Figura 2: Catálogo virtual e replicação de produtos entre vendedores e sites de avaliação.

FedShop

Benchmark de escalabilidade de engines SPARQL federadas.

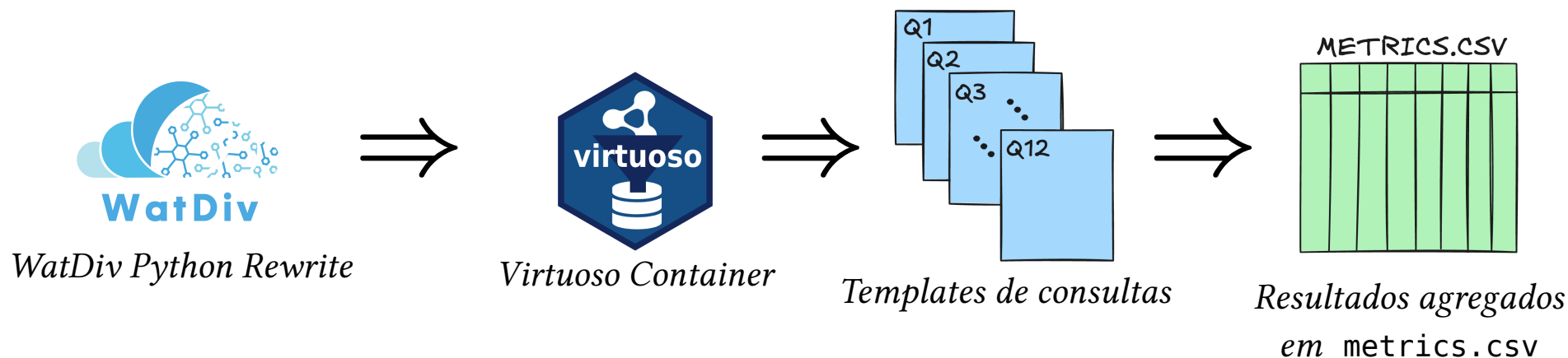
O objetivo é isolar o impacto do tamanho da federação e da estratégia da engine.

Cada execução fixa:

Eixo	Variações consideradas
Consultas	12 <i>templates</i> de consultas
Instâncias	10 valores concretos por template
Escala	10 <i>batches</i> : 20, 40, ..., 200 membros
Fontes	vendedores + sites de avaliação
Tentativas	Repetições controladas

Consulta × Instância × Batch × Tentativa

1. Geração de *n-quads* com WatDiv – Gerador de dados RDF sintéticos usando YAML.
2. Ingestão no *Virtuoso* – Plataforma escalável para banco de dados, integração e LD.
3. Instanciação das consultas – 12 *templates* SPARQL formatados com valores do *dataset*.
4. Execução das *engines* – adapters padronizam chamadas, *timeouts* e coleta de resultados.
5. Agregação das métricas – Tempo de execução, n° de requisições



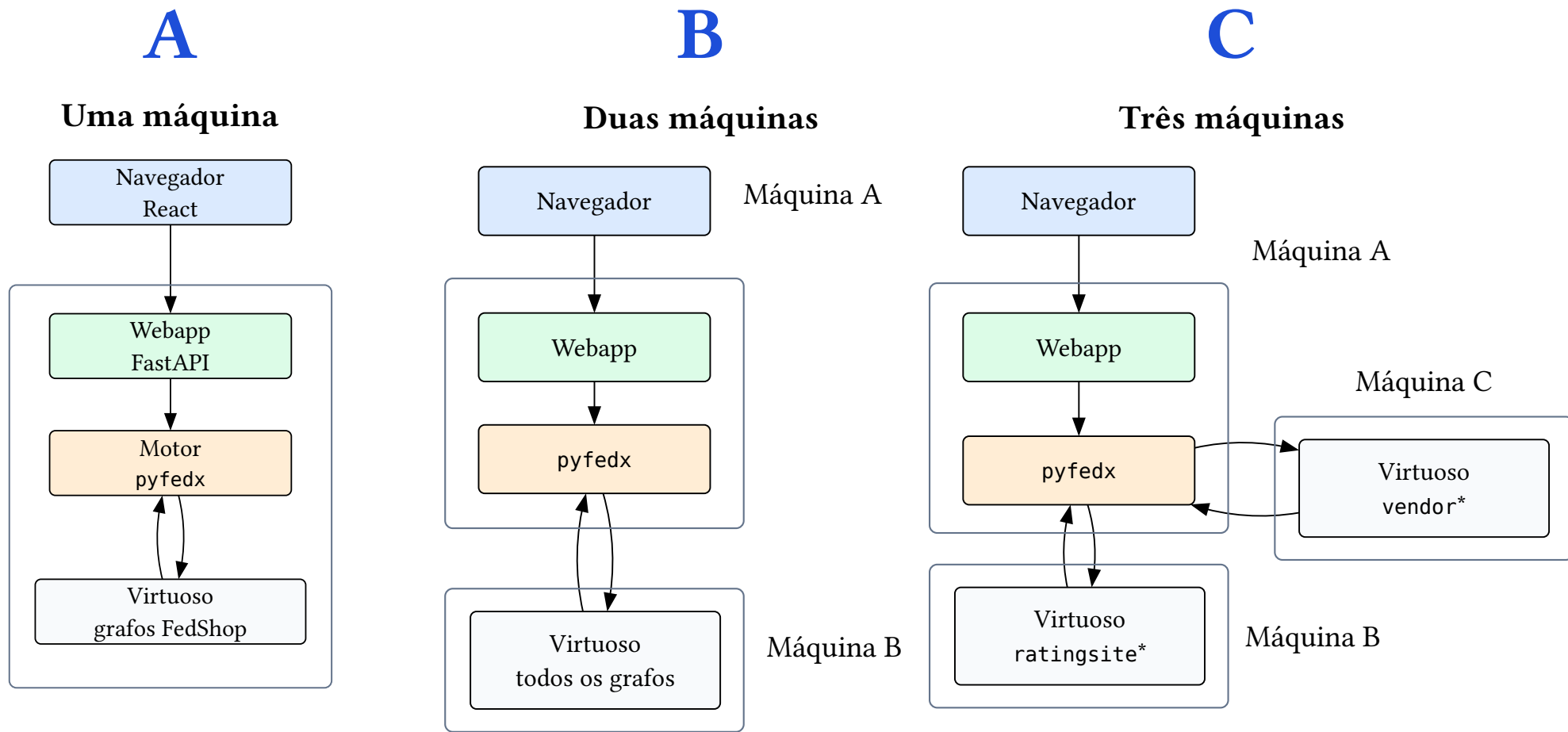
2. Arquitetura

[link para aplicação](#)

* Válido somente para apresentação em aula.

2.2 Arquitetura de Sistema

2. Arquitetura



FEDSHOP_VIRTUOSO_ENDPOINT: um serviço SPARQL remoto.

VENDOR / RATINGSITE + default-graph-uri.

2.3 Arquitetura de Software

2. Arquitetura

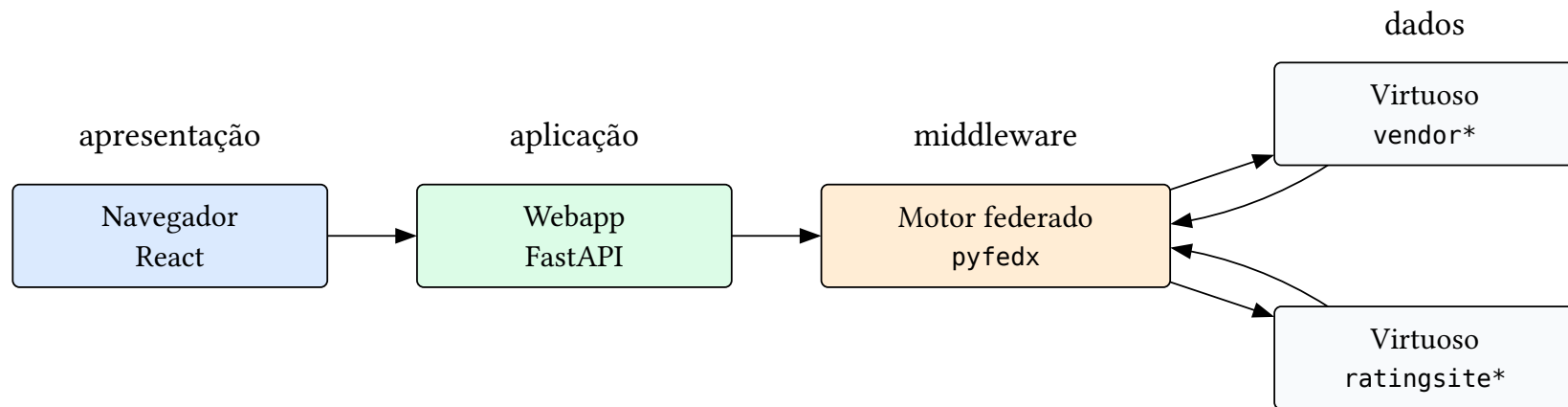


Figura 7: Arquitetura lógica: o motor é servidor para o webapp e cliente dos endpoints SPARQL.

Estágio	Papel no motor
parser.py	SPARQL → Árvore análise sintática
federation.py	ASK multicast + cache
planner.py	ordem das operações
executor.py	SELECT, joins e filtros locais
artifacts.py	resultados, fontes, plano e métricas

Conceitos

- cliente-servidor multicamadas;
- endpoints sem estado;
- ASK() multithreaded com 8 *workers*;
- comunicação em grupo na seleção de fontes;

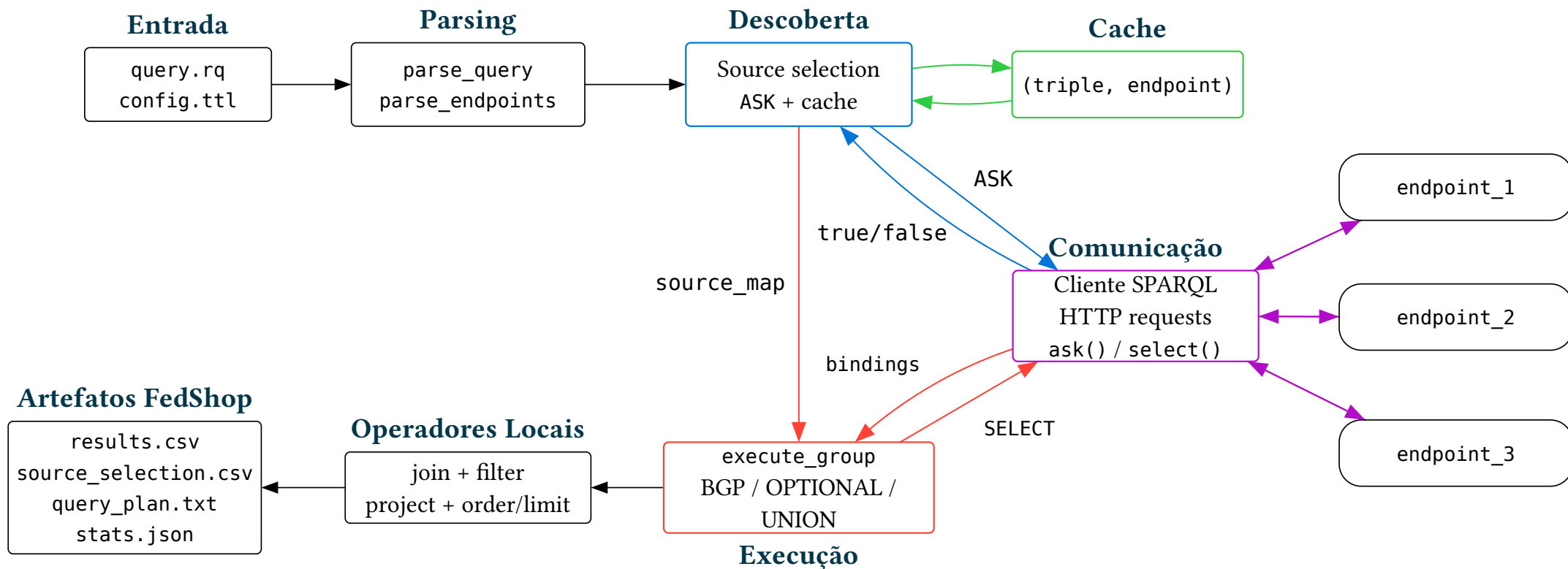


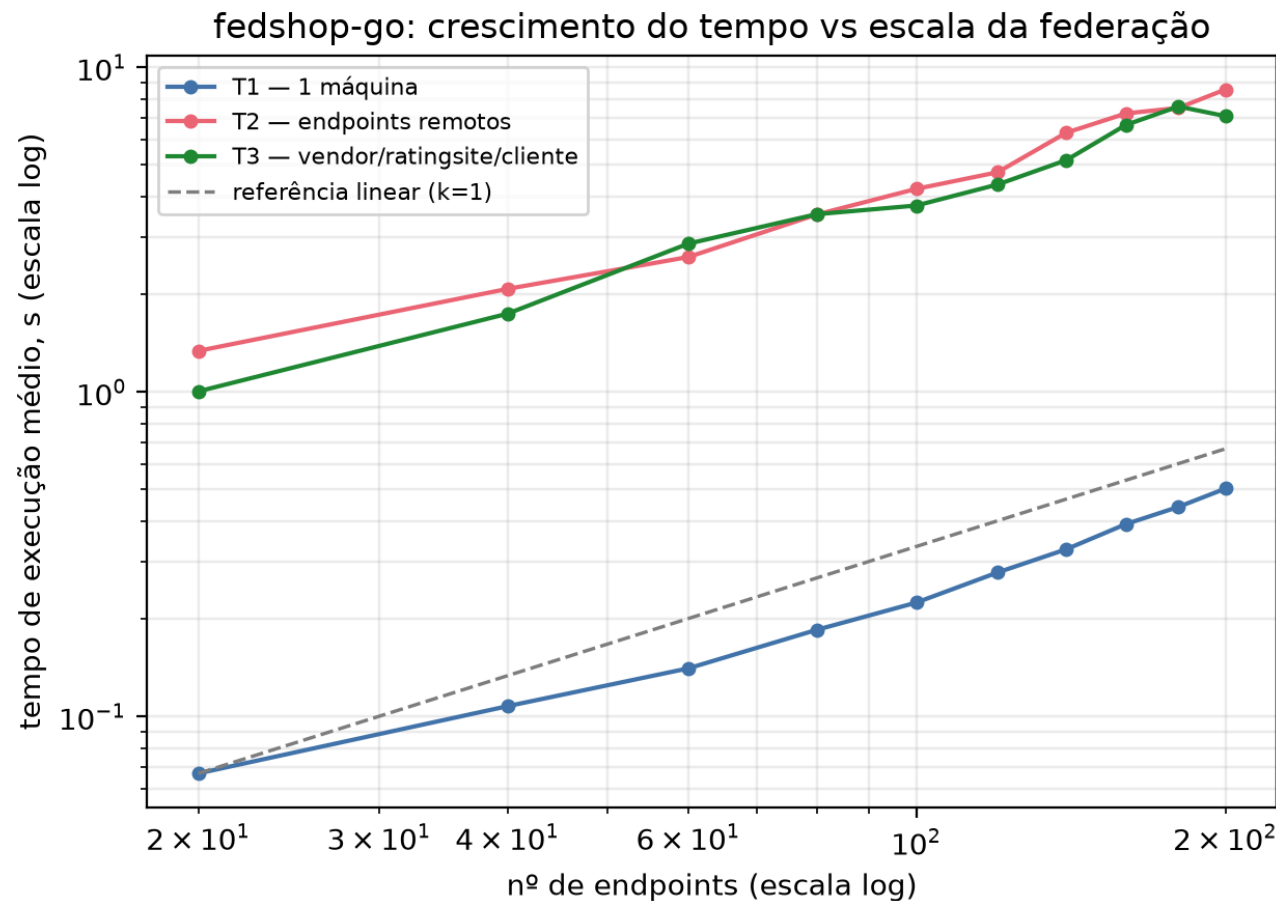
Figura 8: Fluxo interno do pyfedx: seleção de fontes por ASK, execução HTTP e composição local dos resultados.

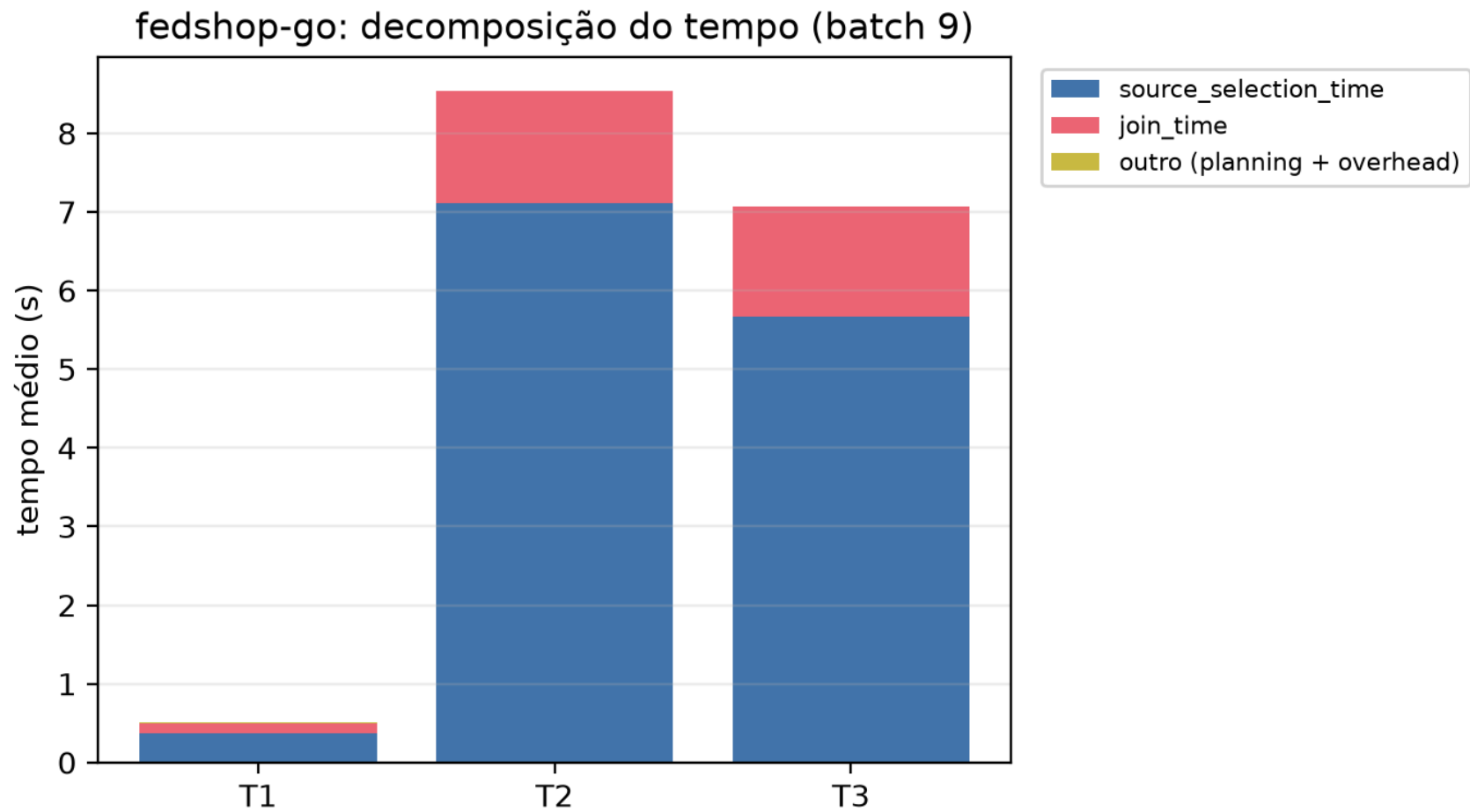
3. Resultados

Topologia	k	R^2	<i>slowdown</i> vs T1	leitura
A — 1 máquina	0,888	0,984	1,00×	sublinear
B — Virtuoso remoto	0,827	0,982	18,05×	sublinear
C — vendor/ratingsite/cliente	0,875	0,986	14,52×	sublinear

Tabela 1: fedshop-go: expoente de crescimento (ajuste log-log de `exec_time` vs nº de endpoints) e *slowdown* no maior lote, por topologia.

* Resultado fortemente dependente das *condições de teste*.
No caso C, uma das máquinas estava mais próxima do cliente...





Referências

- (1) Fernandes, L. C.; Santos Ribeiro, L. dos; Castro, M. V. B. de; Silva Pacheco, L. A. da; Oliveira Sandes, E. F. de. JurisTCU: A Brazilian Portuguese Information Retrieval Dataset with Query Relevance Judgments. *arXiv preprint* **2025**.
- (2) Lv, Y.; Zhai, C. X. When Documents Are Very Long, BM25 Fails!. Em *Proceedings of the 34th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '11)*; 2011; pp 1103–1104. <https://doi.org/10.1145/2009916.2010070>.
- (3) Li, X.; Lipp, J.; Shakir, A.; Huang, R.; Li, J. BMX: Entropy-weighted Similarity and Semantic-enhanced Lexical Search. *arXiv preprint* **2024**.
- (4) Bruch, S.; Gai, S.; Ingber, A. An Analysis of Fusion Functions for Hybrid Retrieval. *arXiv preprint* **2022**.
- (5) Silva Júnior, e. a. Datasets for Portuguese Legal Semantic Textual Similarity. *Language Resources and Evaluation* **2023**.
- (6) —. Análise da Eficácia de Fine-Tuning de Embeddings no Contexto Jurídico Brasileiro. — **202AD**.
- (7) —. Building a Relevance Feedback Corpus for Legal Information Retrieval in the Real-Case Scenario of the Brazilian Chamber of Deputies. Em —; 202AD.
- (8) Lin, J.; Ma, X.; Lin, S.-C.; Yang, J.-H.; Pradeep, R.; Nogueira, R. Pyserini: A Python Toolkit for Reproducible Information Retrieval Research with Sparse and Dense Representations. Em *Proceedings of the 44th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2021)*; 2021; pp 2356–2362.
- (9) Bassani, E. ranx: A Blazing-Fast Python Library for Ranking Evaluation and Comparison. Em *ECIR (2)*; Lecture Notes in Computer Science; Springer, 2022; Vol. 13186, pp 259–264. https://doi.org/10.1007/978-3-030-99739-7_30.